

React + Vite + Django + SQLite

Asset Desk 実装ガイド

申請画面の入力をDjangoへ送り、SQLiteへ保存し、管理画面で確認できるシステムを自力で組み立てる

対象プロジェクト：sylc

作成日：2026年9月3日

学習用・開発環境向け

このガイドでできるようになること

完成済みコードの暗記ではなく、入力欄を追加したときに、保存先・通信処理・管理画面・テストまで自分で変更できることを目標にします。

完成形：ログイン画面から申請種別を選び、申請者情報と申請内容を入力して送信すると、Djangoが検証してSQLiteへ保存します。保存結果はDjango管理画面で検索・確認できます。

全体像



ブラウザはSQLiteを直接操作しません。ReactがJSONをDjangoへ送り、DjangoのORMがSQLを組み立ててSQLiteを操作します。この役割分担がシステムの中心です。

現在の4種類の申請

種別	個別入力項目	保存モデル
PC貸出	氏名、管理番号、使用開始日、利用場所	PcLoanApplication
外部記憶装置貸出	氏名、機器名、容量、場所、貸し出し日	ExternalStorageLoanApplication
LAN機器貸出	機器種別、機器名、利用開始日、返却予定日、利用場所	LanEquipmentLoanApplication
スマートフォン購入	機種、購入日、容量、SIMの有無	SmartphonePurchaseApplication

全申請に共通して、申請者氏名・所属部署・社員番号も手動入力します。

プロジェクトの構造と準備

```
sylc/  
├─ src/  
│   ├── App.jsx          # React画面と送信処理  
│   ├── App.css          # 画面固有の見た目  
│   └─ index.css         # 全体の基本スタイル  
├─ vite.config.js        # 開発サーバーと /api の転送  
├─ package.json          # React側の依存関係・実行コマンド  
└─ backend/  
    ├── manage.py         # Django管理コマンドの入口  
    ├── requirements.txt   # Python依存関係  
    ├── db.sqlite3        # 実データを保持するDBファイル  
    └─ config/  
        ├── settings.py   # DB、アプリ、CSRFなどの全体設定  
        ├── urls.py       # /admin/ と /api/ の入口  
        └─ applications/  
            ├── models.py  # テーブル設計  
            ├── views.py   # JSON受付・検証・保存  
            ├── urls.py    # API URL登録  
            ├── admin.py   # 管理画面の表示設定  
            ├── tests.py   # 自動テスト  
            └─ migrations/ # DB構造の変更履歴
```

最初から作る場合の準備

1 React/Viteを用意

```
npm create vite@latest . -- --template react  
npm install  
npm install lucide-react
```

2 Django用の仮想環境を用意

```
mkdir backend  
cd backend  
py -m venv .env  
.\.env\Scripts\python.exe -m pip install "Django>=5.2,<6.0"
```

3 Djangoプロジェクトとアプリを作る

```
.\.env\Scripts\python.exe -m django startproject config .  
.\.env\Scripts\python.exe manage.py startapp applications
```

既存プロジェクトでは作成コマンドを再実行しません。 同名ファイルを上書きするおそれがあります。現在は依存関係を復元するだけなら `npm install` とPython環境の準備を行います。

Reactで申請画面を組み立てる

`src/App.jsx` は、画面遷移、フォーム定義、入力値の収集、Djangoへの送信を担当します。

1. 項目をデータとして定義する

申請ごとにフォームを複製せず、`REQUEST_TYPES` に「項目の設計図」を置きます。共通の描画処理がこの配列を読み、`input`や`select`を生成します。

```
const REQUEST_TYPES = {
  pc: {
    title: 'PC貸出',
    fields: [
      { name: 'applicantName', label: '氏名', type: 'text', required: true },
      { name: 'managementNumber', label: '管理番号', type: 'text', required: true },
      { name: 'startDate', label: '使用開始日', type: 'date', required: true },
      { name: 'location', label: '利用場所', type: 'text', required: true },
    ],
  },
  // memory, lan, phone も同じ形式で定義
}
```

`name` は通信で使うキーです。後述するDjangoの対応表と一文字単位で一致させます。

2. 申請者情報を共通化する

```
const APPLICANT_FIELDS = [
  { name: 'requesterName', label: '氏名', type: 'text', required: true },
  { name: 'department', label: '所属部署', type: 'text', required: true },
  { name: 'employeeNumber', label: '社員番号', type: 'text', required: true },
]
```

`FormField` が項目定義を受け取り、`type` に応じて`input`または`select`を返します。新しい項目は原則として配列への1件追加で表示できます。

3. React内の画面状態

状態	意味	更新タイミング
<code>userName</code>	ログイン画面で入力した名前	ログインフォーム送信時
<code>selectedType</code>	<code>pc / memory / lan / phone</code>	申請カード選択時

状態	意味	更新タイミング
isComplete	保存成功画面を表示するか	Djangoが201を返した後
isSubmitting	二重送信を防ぐ通信中フラグ	送信開始から終了まで
submitError	利用者へ見せる失敗理由	通信または検証失敗時

現状のログインは画面上のデモです。未入力を防ぐだけで、Djangoのユーザー認証は行っていません。パスワードは送信・保存されません。本番利用ではDjango認証、HTTPS、ログアウト、権限設計が必要です。

Djangoでデータベースを設計する

backend/applications/models.py のモデルは、Pythonで書くデータベースの設計図です。DjangoがこれをSQLiteのテーブルに変換します。

共通項目を抽象モデルにする

```
class BaseApplication(models.Model):
    requester_name = models.CharField('申請者氏名', max_length=100)
    department = models.CharField('所属部署', max_length=100)
    employee_number = models.CharField('社員番号', max_length=30)
    created_at = models.DateTimeField('作成日時', auto_now_add=True)
    updated_at = models.DateTimeField('更新日時', auto_now=True)

    class Meta:
        abstract = True
```

abstract = True により、BaseApplication専用テーブルは作られません。各申請モデルに共通列を引き継ぐための設計部品になります。

申請種別ごとのモデルを作る

```
class PcLoanApplication(BaseApplication):
    applicant_name = models.CharField('氏名', max_length=100)
    management_number = models.CharField('管理番号', max_length=50)
    start_date = models.DateField('使用開始日')
    location = models.CharField('利用場所', max_length=200)

class ExternalStorageLoanApplication(BaseApplication):
    applicant_name = models.CharField('氏名', max_length=100)
    device_name = models.CharField('機器名', max_length=100)
    capacity = models.CharField('容量', max_length=50)
    location = models.CharField('場所', max_length=200)
    loan_date = models.DateField('貸し出し日')
```

Djangoフィールド	SQLiteでの意味	用途
CharField	文字列列	氏名、番号、場所など
DateField	日付列	使用開始日、貸し出し日など
DateTimeField	日時列	作成・更新日時
TextChoices	許容値を限定	LAN機器種別、容量、SIM有無

モデル自身に業務ルールを書く

```
def clean(self):
    super().clean()
    if self.start_date and self.return_date \
        and self.return_date < self.start_date:
        raise ValidationError({
            'return_date': '返却予定日は利用開始日以降にしてください。'
        })
```

この例はLAN機器の返却予定日が開始日より前になるのを防ぎます。画面側だけでなくモデル側にも書くことで、管理画面や別処理から保存するときも同じルールが働きます。

マイグレーションで設計をDBへ反映する

モデルを書いただけではSQLiteのテーブルは変わりません。設計変更を履歴にし、その履歴をDBへ適用するのがマイグレーションです。



```
cd backend
.\.venv\Scripts\python.exe manage.py makemigrations
.\.venv\Scripts\python.exe manage.py migrate
```

- **makemigrations** : モデルとの差分を読み、migrationsフォルダに変更手順を作る。
- **migrate** : 未適用の変更手順をSQLiteへ実行する。

```
# 適用状況
.\.venv\Scripts\python.exe manage.py showmigrations

# モデルとmigrationに差分がないか
.\.venv\Scripts\python.exe manage.py makemigrations --check --dry-run
```

db.sqlite3だけを直接編集しない：チームで同じ構造を再現するには、モデルとmigrationをGitで共有します。通常、個人の実データを含むdb.sqlite3は共有対象から外します。

SQLiteとSQLの位置づけ

SQLはデータの追加・検索・更新・削除を指示する言語です。**SQLite**はSQLを理解し、1個のファイルにデータを保存するデータベースソフトです。このシステムではDjango ORMがSQLを自動生成するため、通常はSQLを手書きしません。

URLと保存処理を実装する

「URLを登録する」とは、届いたURLと実行するPython関数を対応付けることです。

URLを二段階で接続する

```
# backend/config/urls.py
urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('applications.urls')),
]

# backend/applications/urls.py
urlpatterns = [
    path('csrf/', csrf_token, name='csrf-token'),
    path('applications/', create_application, name='create-application'),
]
```

結果として `/api/csrf/` は `csrf_token` 関数へ、`/api/applications/` は `create_application` 関数へ到達します。末尾のスラッシュもReact側と合わせます。

React名とDjango名を対応付ける

```
APPLICATION_SETTINGS = {
    'pc': {
        'model': PcLoanApplication,
        'fields': {
            'applicantName': 'applicant_name',
            'managementNumber': 'management_number',
            'startDate': 'start_date',
            'location': 'location',
        },
    },
    # memory, lan, phone も定義
}
```

JavaScriptではcamelCase、Pythonではsnake_caseが一般的です。この辞書は表記を変換し、申請種別ごとに保存モデルを選びます。

JSONを検証して保存する

```
@require_POST
def create_application(request):
    payload = json.loads(request.body or '{}')
    request_type = payload.get('requestType')
    settings = APPLICATION_SETTINGS.get(request_type)

    model_values = {
        'requester_name': payload.get('requesterName'),
        'department': payload.get('department'),
        'employee_number': payload.get('employeeNumber'),
    }
    for frontend_name, model_name in settings['fields'].items():
        model_values[model_name] = payload.get(frontend_name)

    application = settings['model'](**model_values)
    application.full_clean()
    with transaction.atomic():
        application.save()

    return JsonResponse({'id': application.pk}, status=201)
```

1. JSONをPythonの辞書へ変換する。
2. requestTypeからモデルと項目対応を選ぶ。
3. full_clean()で必須、長さ、日付、選択肢、独自ルールを検証する。
4. transaction.atomic()内で保存し、途中失敗時の中途半端な更新を防ぐ。
5. 成功時はHTTP 201とIDを返す。入力不正は400を返す。

CSRFで不正送信を防ぐ

CSRFは、利用者がログイン済みのサイトに対して、別サイトから意図しない操作を送らせる攻撃です。DjangoはPOSTに秘密の照合値を要求して防ぎます。

正常な送信の順序

1. Reactが GET /api/csrf/ を呼ぶ。
2. Djangoがブラウザへcsrftoken Cookieを発行する。
3. ReactがCookieを読み、POSTのX-CSRFTokenヘッダーへ入れる。
4. DjangoがCookieとヘッダーを照合し、一致すれば保存処理へ進む。

```
# views.py
@require_GET
@ensure_csrf_cookie
def csrf_token(request):
    return JsonResponse({'message': 'CSRF Cookieを発行しました。'})

# settings.py
CSRF_TRUSTED_ORIGINS = [
    'http://localhost:5173',
    'http://127.0.0.1:5173',
]
```

信頼済みOriginとは：「このURLで動くフロントエンドからの安全なPOSTを受け入れる」という許可リストです。ホスト名、ポート、http/httpsのどれかが違えば別Originです。

React側のトークン取得

```
async function getCsrftoken() {
  const existingToken = getCookie('csrftoken')
  if (existingToken) return existingToken

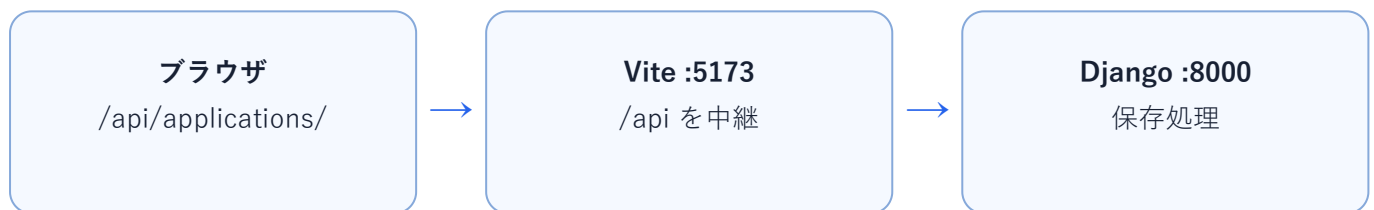
  const response = await fetch('/api/csrf/', {
    credentials: 'same-origin',
  })
  if (!response.ok) {
    throw new Error('セキュリティ情報を取得できませんでした。')
  }
  return getCookie('csrftoken')
}
```

`csrf_exempt` で保護を外す方法は採用していません。原因を直してCSRF検証を通す方が、安全な設計を維持できます。

ViteからDjangoへ通信を転送する

開発中、画面はViteの5173番、保存処理はDjangoの8000番で動きます。Viteのproxyはブラウザの **/api** 通信をDjangoへ中継します。

```
// vite.config.js
export default defineConfig({
  plugins: [react(), sites()],
  server: {
    proxy: {
      '/api': {
        target: 'http://127.0.0.1:8000',
        changeOrigin: true,
      },
    },
  },
})
```



- ReactにDjangoの完全URLを埋め込まず、**/api/...** と書ける。
- 開発時の異なるポート間通信を扱いやすくする。
- **vite.config.js** を変更した後はViteを再起動する。

proxyは開発用です。 本番ではWebサーバーやホスティング環境で、フロントエンドとDjangoへの経路を別途設定します。

submitRequestを非同期処理にする

非同期処理とは、通信完了を待ちながらブラウザ全体を再読み込みせず、成功・失敗に応じて画面だけを更新する処理です。

```
const submitRequest = async (event) => {
  event.preventDefault()
  setIsSubmitting(true)
  setSubmitError('')

  try {
    const formValues = Object.fromEntries(new FormData(event.currentTarget))
    const csrfToken = await getCsrftoken()
    const response = await fetch('/api/applications/', {
      method: 'POST',
      credentials: 'same-origin',
      headers: {
        'Content-Type': 'application/json',
        'X-CSRFToken': csrfToken,
      },
      body: JSON.stringify({
        requestType: selectedType,
        ...formValues,
      }),
    })

    const data = await response.json()
    if (!response.ok) throw new Error(getSubmissionError(data))
    setSubmissionId(data.id)
    setIsComplete(true)
  } catch (error) {
    setSubmitError(error.message)
  } finally {
    setIsSubmitting(false)
  }
}
```

コード	役割
preventDefault()	フォーム標準のページ再読み込みを止める
FormData	name属性が付いた入力値をまとめて収集する
await fetch(...)	HTTP応答が返るまで、この関数内の次処理を待つ
response.ok	HTTP 200番台かを判定する
try / catch	通信・JSON・サーバーエラーを画面表示へ変換する
finally	成功・失敗の両方で通信中状態を解除する

ボタンを押しただけでは完了扱いにせず、Djangoが保存して201を返した後だけ完了画面へ進めるのが重要です。

Django管理画面を有効にする

管理画面はDjango標準機能です。URL、標準アプリ、モデル登録の3点がつながって動きます。

```
# config/urls.py
path('admin/', admin.site.urls)

# applications/admin.py
@admin.register(PcLoanApplication)
class PcLoanApplicationAdmin(BaseApplicationAdmin):
    list_display = (
        'id', 'management_number', 'applicant_name',
        'department', 'start_date', 'location', 'created_at',
    )
    search_fields = ('requester_name', 'management_number', 'location')
    list_filter = ('start_date', 'department')
```

`@admin.register` がモデルを管理画面へ登録します。list_displayは一覧列、search_fieldsは検索対象、list_filterは絞り込み項目です。

管理者を作成して確認する

```
cd backend
.\.venv\Scripts\python.exe manage.py createsuperuser
.\.venv\Scripts\python.exe manage.py runserver
```

ブラウザで <http://127.0.0.1:8000/admin/> を開きます。入力した管理者のユーザー名、メールアドレス、ハッシュ化されたパスワードは、標準認証用テーブルとして `backend/db.sqlite3` に保存されます。

パスワード検証もDjango標準機能です。 `AUTH_PASSWORD_VALIDATORS` により、ユーザー名との類似、8文字未満、よくある文字列、数字だけなどが検査されます。平文パスワードはDBへ保存されません。

起動と動作確認

DjangoとViteは別々のプロセスです。PowerShellを2つ開いて両方を起動します。

PowerShell 1 : Django

```
cd C:\Users\sylc0276\takino\sylc\backend
.\.venv\Scripts\python.exe manage.py migrate
.\.venv\Scripts\python.exe manage.py runserver
```

PowerShell 2 : Vite

```
cd C:\Users\sylc0276\takino\sylc
npm install
npm run dev
```

表示されたViteのURL（通常は <http://localhost:5173/>）をブラウザで開きます。

画面からの確認手順

- ログイン名とパスワードを入力してログインする。
- PC貸出などの申請カードを選ぶ。
- 申請者情報と申請内容をすべて入力する。
- 「この内容で申請する」を押す。
- 完了画面に受付IDが表示されることを確認する。
- Django管理画面を開き、該当申請一覧に同じ内容があることを確認する。

ブラウザ開発者ツールで通信を確認

Network欄	期待値	意味
GET /api/csrf/	200	CSRF Cookieの準備成功
POST /api/applications/	201	検証・保存成功
Response	id, requestType, message	Djangoからの保存結果

Network欄	期待値	意味
Request Payload	入力したJSON	Reactが送った値

自動テストと品質確認

目視確認だけでなく、保存と拒否条件をプログラムで繰り返し確認します。

```
cd backend
.\.venv\Scripts\python.exe manage.py check
.\.venv\Scripts\python.exe manage.py test
.\.venv\Scripts\python.exe manage.py makemigrations --check --dry-run

cd ..
npm run lint
npm run build
```

確認	検出する問題
manage.py check	Django設定やモデル定義の基本的な不整合
manage.py test	4申請の保存、必須項目、日付ルール、CSRF拒否
makemigrations --check	モデル変更後のmigration作り忘れ
npm run lint	JavaScript/Reactコードの規約違反や典型的な誤り
npm run build	本番用React資産を生成できるか

良いテストの考え方：成功例だけでなく、管理番号がない、CSRFトークンがない、返却日が開始日より前など「保存してはいけない例」も確認します。

エラーの切り分け方

症状	主な原因	確認・対処
sites-vite-pluginを解決できない	node_modulesに依存パッケージがない	プロジェクト直下で <code>npm install</code> 。package.jsonも確認。
セキュリティ情報を取得できない	Django停止、proxy未反映、URL違い	runserverを確認。Viteを再起動。NetworkでGET /api/csrf/を見る。
申請を保存できない／403	CSRFトークン不一致、Origin未許可	CookieとX-CSRFToken、CSRF_TRUSTED_ORIGINSを確認。
申請を保存できない／400	必須値、選択肢、日付などの検証失敗	NetworkのResponse内fieldsを確認。キー対応を照合。
no such table	migration未適用	<code>manage.py migrate</code> を実行。
管理画面にモデルがない	admin.pyで未登録	<code>@admin.register(...)</code> とINSTALLED_APPSを確認。
画面変更が反映されない	違うGitブランチ、キャッシュ	<code>git branch --show-current</code> 、Vite再起動、再読み込み。

切り分けの順番

1. Djangoのターミナルに例外やHTTPステータスが出ているか。
2. ブラウザのNetworkで、どのリクエストが何番で失敗したか。
3. Response本文にDjangoの検証理由があるか。
4. Request Payloadのキー名と値が期待どおりか。
5. モデル、対応表、Reactのname属性が一致しているか。

画面のメッセージだけで推測せず、HTTPステータスと送受信内容を見れば問題の層を絞れます。

新しい申請項目を追加する練習

例としてPC貸出に「返却予定日」を追加する場合、次の順序で変更します。

1. **画面**：`REQUEST_TYPES.pc.fields` に`returnDate`を追加。
2. **モデル**：`PcLoanApplication` に`return_date`を追加。
3. **対応表**：`'returnDate': 'return_date'` を追加。
4. **DB**：`makemigrations`と`migrate`を実行。
5. **管理画面**：`list_display`や`list_filter`に追加。
6. **テスト**：正常値の保存と不正値の拒否を追加。
7. **確認**：Djangoテスト、`lint`、`build`、画面送信、管理画面を順に確認。

変更の一本線：`input`の`name` → 送信JSONのキー → `APPLICATION_SETTINGS` → モデルのフィールド → SQLiteの列。値が消えたときは、この線を左からたどります。

自力実装チェックリスト

- 各ファイルの責任を説明できる。
- 新しい入力欄を追加し、対応するモデル列を作れる。
- migrationが必要な理由を説明し、適用できる。
- URLからviewまでの到達経路を説明できる。
- CSRF Cookieとヘッダーの関係を説明できる。
- HTTP 201、400、403の違いをNetworkで判断できる。
- 管理画面と自動テストで保存を確認できる。

用語集と本番化の注意

用語	このシステムでの意味
React	状態に応じてログイン、一覧、入力、完了画面を描画するライブラリ。
Vite	Reactの開発サーバーとビルドを担当し、開発中は/apiをDjangoへ転送。
Django	URL受付、CSRF、入力検証、ORM、管理画面、認証基盤を提供。
モデル	保存項目の型、長さ、選択肢、ルールを表すPythonクラス。
ORM	Pythonオブジェクト操作をSQLへ変換する仕組み。
SQLite	backend/db.sqlite3という1ファイルで動く開発向けDB。
API	ReactとDjangoがJSONでデータを受け渡す窓口。
CSRF	第三者サイトからの意図しないPOSTを防ぐ仕組み。
migration	モデル変更をDB構造へ再現可能に適用する履歴。

本番環境へ進む前に

- `DEBUG = False`、安全な`SECRET_KEY`、適切な`ALLOWED_HOSTS`を設定する。
- HTTPSを必須にし、CSRF/Cookieの本番設定を見直す。
- 画面だけのログインをDjango認証へ置き換え、権限と操作履歴を設計する。
- 個人情報の保存期間、閲覧権限、バックアップ、削除手順を定める。
- 規模に応じてPostgreSQLなどのサーバー型DBを検討する。
- Djangoの開発サーバーではなく、本番用構成で配信する。

この実装は学習・開発用の土台です。特に現在のログイン画面は本人確認をしていないため、そのまま社内本番システムとして公開しないでください。

最後に：実装を理解する最短ルート

まずPC貸出だけを対象に、入力 → JSON → view → model → SQLite → adminの一本を自分で再現してください。一本が理解できれば、他の申請は同じ構造の横展開です。機能を追加するたびに正常系と異常系のテストも増やすと、壊れにくいシステムになります。

